Quadruple Generation using Lex and Yacc

Intermediate code for the statement X = A + B

1. Aim

To write a Lex and Yacc program that reads an assignment statement such as X = A + B and generates its intermediate code in the form of quadruples.

2. What is a Quadruple?

Intermediate code is often stored as quadruples. Each quadruple has 4 fields:

`(operator, arg1, arg2, result)`

For X = A + B the quadruples are:

```
(+, A, B, t1)
(=, t1, -, X)
```

3. Algorithm

1. The lexer (Lex) reads the input and returns tokens: ID, NUM, NL and single characters such as = + - * / ( ).
2. The parser (Yacc) matches the tokens against the grammar rules.
3. When an operator rule is reduced (for example expr + expr), create a new temporary (t1, t2, ...) and store a quadruple (op, arg1, arg2, temp).
4. When the assignment rule ID = expr is reduced, store the quadruple (=, expr, -, ID).
5. After the whole input is parsed, print the quadruple table.

4. Program

4.1 quad.l (lexer)

The lexer reads characters and turns them into tokens.

```
%{
#include <stdio.h>
#include <string.h>
#include "y.tab.h"
%}

%%
[a-zA-Z_][a-zA-Z0-9_]*   { yylval.str = strdup(yytext); return ID; }
[0-9]+                   { yylval.str = strdup(yytext); return NUM; }
[ \t]+                   ;
\n                       { return NL; }
.                        { return yytext[0]; }
%%

int yywrap(void) { return 1; }
```

Line	Meaning
`#include "y.tab.h"`	Brings in the token names (ID, NUM, NL) and yylval from Yacc.
`[a-zA-Z_]...`	Matches a variable name such as A, B or X. It stores the text in yylval.str and returns ID.
`[0-9]+`	Matches a number and returns NUM.
`[ \t]+ ;`	Ignores spaces and tabs.
`\n`	Returns NL to mark the end of a statement.
`.`	Returns any other single character (=, +, (and so on) as itself.
`yywrap()`	Returning 1 tells Lex that input ends at end of file.

4.2 quad.y (parser and quadruple generator)

Part 1: C declarations

```
%{
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

int yylex(void);
void yyerror(const char *s);

struct quad {
    char op[4];
    char arg1[32];
    char arg2[32];
    char result[32];
} q[200];

int qcount = 0;   // number of quadruples stored
int tcount = 0;   // number of temporaries created

char *newtemp(void) {            // makes t1, t2, t3 ...
    char *t = malloc(16);
    sprintf(t, "t%d", ++tcount);
    return t;
}

void addquad(const char *op, const char *a1, const char *a2, const char *res) {
    strcpy(q[qcount].op, op);
    strcpy(q[qcount].arg1, a1);
    strcpy(q[qcount].arg2, a2);
    strcpy(q[qcount].result, res);
    qcount++;
}
%}
```

- `struct quad q[200]` is the quadruple table.
- `newtemp()` creates temporary variable names.
- `addquad()` appends one row to the table.

Part 2: Yacc declarations

```
%union { char *str; }

%token <str> ID NUM
%token NL
%type  <str> expr

%left '+' '-'
%left '*' '/'
```

- `%union` says each grammar symbol carries a char * (a name or temporary).
- `%token <str> ID NUM` means these tokens carry that string value.
- `%type <str> expr` means an expr also carries a name. For A+B it carries t1.
- `%left` gives associativity and precedence. Operators listed later bind tighter, so * and / bind tighter than + and -.

Part 3: Grammar rules and actions

```
%%

program : program line
        | line
        ;

line    : ID '=' expr NL   { addquad("=", $3, "-", $1); }
        | NL
        | error NL         { yyerrok; }
        ;

expr    : expr '+' expr    { $$ = newtemp(); addquad("+", $1, $3, $$); }
        | expr '-' expr    { $$ = newtemp(); addquad("-", $1, $3, $$); }
        | expr '*' expr    { $$ = newtemp(); addquad("*", $1, $3, $$); }
        | expr '/' expr    { $$ = newtemp(); addquad("/", $1, $3, $$); }
        | '(' expr ')'     { $$ = $2; }
        | ID               { $$ = $1; }
        | NUM              { $$ = $1; }
        ;

%%
```

In an action, `$$` is the value of the left-hand side, and `$1`, `$2`, `$3` are the values of the right-hand symbols in order.

For `expr '+' expr`:

- `$1` is the left operand and `$3` is the right operand.
- `$$ = newtemp()` creates a new temporary, say t1.
- `addquad("+", $1, $3, $$)` stores (+, A, B, t1).
- expr now has the value t1, so a bigger expression can use it.

For `ID '=' expr NL`:

- `$1` is X and `$3` is the expression's result.
- `addquad("=", $3, "-", $1)` stores (=, t1, -, X).

`expr : ID` just passes the variable name up (`$$ = $1`). `error NL` skips a bad line so parsing can continue.

Part 4: C code (main)

```
void yyerror(const char *s) {
    fprintf(stderr, "Error: %s\n", s);
}

int main(void) {
    printf("Enter statement(s), e.g. X=A+B (Ctrl+D to finish):\n");
    yyparse();

    printf("\n%-6s %-8s %-8s %-8s %-8s\n", "Index", "Operator", "Arg1", "Arg2",
"Result");
    printf("-------------------------------------------------\n");
    for (int i = 0; i < qcount; i++)
        printf("%-6d %-8s %-8s %-8s %-8s\n", i, q[i].op, q[i].arg1, q[i].arg2,
q[i].result);
    return 0;
}
```

`yyparse()` runs the parser, and the table is printed once the whole input has been read.

5. Trace of X=A+B

Step	Tokens seen	What happens
1	`ID(X)  =`	The parser waits for an expression.
2	`ID(A)`	expr -> ID, so expr = "A".
3	`+`	The parser shifts.
4	`ID(B)`	expr -> ID, so expr = "B".
5	`reduce expr + expr`	Makes t1 and stores (+, A, B, t1).
6	`NL`	Reduces line and stores (=, t1, -, X).

6. Compile and Run

```
yacc -d quad.y
lex quad.l
gcc lex.yy.c y.tab.c -o quad
./quad
```

If your system uses flex or bison, use `flex quad.l` and `bison -y -d quad.y` instead.

7. Output

Input X=A+B, press Enter, then Ctrl+D:

```
Enter statement(s), e.g. X=A+B (Ctrl+D to finish):
X=A+B

Index  Operator Arg1     Arg2     Result
-----------------------------------------------
0      +        A        B        t1
1      =        t1       -        X
```

A longer statement such as X=A+B*C also works. Multiplication is done first because of the precedence rules:

```
Index  Operator Arg1     Arg2     Result
-----------------------------------------------
0      *        B        C        t1
1      +        A        t1       t2
2      =        t2       -        X
```

8. Result

The Lex and Yacc program was written and executed successfully. It generates the quadruples for the statement X = A + B.